



KasKold Security Architecture

Hardware and Vault custody boundaries, watch-only Companion, key lifecycle, and authorization

Assurance is layered, not absolute: automated QA and reproducible builds do not make consumer hardware tamper resistant or formally verified.

1. Trust and custody boundaries

Component	Security role
KasKold Hardware	Offline key custody, independent parsing/review/signing, QR/SD exchange
KasKold Vault Web	Software custody/review/signing; no blockchain watcher/node/broadcast path
KasKold Vault Android / iOS	Dedicated network-isolated software signer applications
KasKold Companion Web / Android / iOS	Strictly watch-only online state, transaction construction, finalization and broadcast

2. Secret boundary

- Companion does not depend on hot-wallet, vault-runtime, or offline-signer and cannot create/restore/sign wallets.
- Vault applications reuse the shared Rust vault-runtime and hardened signing implementation; UI layers do not reimplement cryptography.
- Ordinary signer operations use constrained Rust state and do not expose private-key bytes as a side effect of signing.

3. Entropy and key lifecycle

Hardware seed creation uses the documented checked hardware entropy pipeline. Vault software wallet creation uses the platform CSPRNG through Rust. Transient buffers are zeroized where the implementation owns them.

4. Transaction and QR authorization

- KSPT v4 multi-frame assembly is session-bound and rejects conflicting/mixed framing.
- Hardware and Vault parse and review signing intent independently of Companion.
- Vault scanning enters review state; signing requires a separate explicit approval action.

5. Compatibility identifiers and residual risks

Some legacy wire and cryptographic domain identifiers intentionally retain historical KasSigner spelling because renaming them would alter compatibility or cryptographic semantics.

- Residual risks include physical/fault attacks, compromised online clients/nodes/toolchains, weak backups/passwords, and unqualified hardware variants.